

Week -05

```
#include <stdio.h>
```

```
#define MAX_TASKS 10
```

```
#define MAX_TIME 20
```

```
typedef struct {
```

```
    int id;
```

```
    int C;
```

```
    int P;
```

```
    int next_arrival;
```

```
    int remaining_time;
```

```
} Task;
```

```
void sort_by_period(Task tasks[], int n) {
```

```
    Task temp;
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (tasks[i].P > tasks[j].P) {
```

```
                temp = tasks[i];
```

```
                tasks[i] = tasks[j];
```

```
                tasks[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    Task tasks[MAX_TASKS];
```

```

int gantt[MAX_TIME]; // store execution timeline


printf("Enter number of tasks: ");
scanf("%d", &n);


// Input
for (int i = 0; i < n; i++) {
    tasks[i].id = i + 1;

    printf("Enter execution time (C) and period (P) for Task %d: ", i + 1);
    scanf("%d %d", &tasks[i].C, &tasks[i].P);
}


sort_by_period(tasks, n);


// Initialize
for (int i = 0; i < n; i++) {
    tasks[i].next_arrival = 0;
    tasks[i].remaining_time = 0;
}


printf("\nScheduling Timeline:\n");


for (int time = 0; time < MAX_TIME; time++) {

    // arrivals
    for (int i = 0; i < n; i++) {
        if (time == tasks[i].next_arrival) {
            tasks[i].remaining_time = tasks[i].C;
            tasks[i].next_arrival += tasks[i].P;
        }
    }
}

```

```

// select task

int current_task = -1;
for (int i = 0; i < n; i++) {
    if (tasks[i].remaining_time > 0) {
        current_task = i;
        break;
    }
}

// execute

if (current_task != -1) {
    printf("Time %2d: Task %d running\n",
        time, tasks[current_task].id);
    tasks[current_task].remaining_time--;
    gantt[time] = tasks[current_task].id;
} else {
    printf("Time %2d: CPU Idle\n", time);
    gantt[time] = 0;
}
}

// ♦ Gantt Chart

printf("\nGantt Chart:\n|");
for (int i = 0; i < MAX_TIME; i++) {
    if (gantt[i] == 0)
        printf(" Idle |");
    else
        printf(" T%d |", gantt[i]);
}

```

```

printf("\n0");

for (int i = 1; i <= MAX_TIME; i++) {

    printf("  %d", i);

}

printf("\n");

return 0;

}

```

OUTPUT:

```

C:\Users\student\Desktop\realmonotic.exe
Enter number of tasks: 3
Enter execution time (C) and period (P) for Task 1: 3 20
Enter execution time (C) and period (P) for Task 2: 2 5
Enter execution time (C) and period (P) for Task 3: 2 10

Scheduling Timeline:
Time 0: Task 2 running
Time 1: Task 2 running
Time 2: Task 3 running
Time 3: Task 3 running
Time 4: Task 1 running
Time 5: Task 2 running
Time 6: Task 2 running
Time 7: Task 1 running
Time 8: Task 1 running
Time 9: CPU Idle
Time 10: Task 2 running
Time 11: Task 2 running
Time 12: Task 3 running
Time 13: Task 3 running
Time 14: CPU Idle
Time 15: Task 2 running
Time 16: Task 2 running
Time 17: CPU Idle
Time 18: CPU Idle
Time 19: CPU Idle

Gantt Chart:
| T2 | T2 | T3 | T3 | T1 | T2 | T2 | T1 | T1 | Idle | T2 | T2 | T3 | T3 | Idle | T2 | T2 | Idle | Idle | Idle |
0   1   2   3   4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20

Process returned 0 (0x0)   execution time : 11.959 s
Press any key to continue.

```